ID	FOL	Norm. FOL	TYPE	Change
E33	$foo(x)$	$foo(x)$	REF	–
E34	$y.foo(x)$	$y.foo(x)$	REF	–
E35,36	$\text{instanceOf}_T(x)$	$\text{instanceOf}_T(x)$	BOOL	–
E37	$\text{ite}(x > 0, x, y)$	$\text{ite}(x > 0, x, y)$	INT	–
E38	$\text{ite}(true, x, null)$	$\neg true \vee x$	BOOL	✓
E39,42-46	$\exists e\,(e \in args[0] \wedge e = null)$	$\exists e\,(e \in args[0] \wedge e = null)$	BOOL	–
E40	$\forall e\,(e \in args[0] \rightarrow e = null)$	$\forall e\,(e \in args[0] \rightarrow e = null)$	BOOL	–
E41	$\neg\exists e\,(e \in args[0] \wedge e = null)$	$\neg\exists e\,(e \in args[0] \wedge e = null)$	BOOL	–

5.2 Modeling Assumptions and Abstraction

Abstraction of Stream and Lambda Semantics. The normalization rules (SN5)–(SN8) (see Subsection 5.1.1) rewrite different syntactic variants of stream constructions and lambda expressions into a uniform canonical form used by the translation pipeline.
The stream normalization rules (SN5) and (SN6) abstract from the concrete mechanism used to construct a stream. In Java, streams may be obtained via static factory methods (e.g., `Arrays.stream(a)`) or via instance methods (e.g., `a.stream()`) [18]. Since the evaluation framework does not model operational aspects of stream creation (such as laziness, parallelism, or intermediate pipeline stages), all supported stream constructions are interpreted extensionally as ranging over the elements of the underlying collection or array. Consequently, these rewritings are semantics-preserving under the specification-oriented abstraction adopted in this work and ensure a uniform syntactic pattern required for the subsequent quantifier-based translation.

The lambda canonicalisation rules (SN7) and (SN8) rewrite method references into syntactically uniform lambda expressions. The transformation (SN7) is semantics-preserving even under standard Java execution semantics. The method reference `Objects::isNull` is equivalent to the lambda expression `e -> Objects.isNull(e)`, and since `Objects.isNull(e)` is defined as `e == null`, both forms denote the same total, side-effect free Boolean predicate [19, 20].
In contrast, (SN8) is not strictly semantics-preserving under full Java operational semantics. The method reference `null::equals` corresponds to the lambda `e -> null.equals(e)`, which in executable Java would result in a `NullPointerException` [21]. However, within the context of specification-level assertions, method references are interpreted extensionally, i.e., in terms of the predicate they denote rather than their runtime invocation behavior.

In this abstract interpretation, occurrences of `null::equals` are naturally understood as expressing the null-check predicate `e == null`. The evaluation framework operates on isolated assertions and compares their logical meaning rather than executable program behavior. Lambda expressions used in stream predicates are assumed to be pure, total, and side-effect free. Under these assumptions, rewriting `null::equals` to the canonical form `e -> e == null` preserves the intended specification meaning and enables uniform translation into first-order logic.

5.3 Assertion Comparison Framework

After translating Java assertions into their canonical FOL representation, the tool determines the semantic relationship between an *expected* assertion and a *generated* assertion. For each

pair of conditions, exactly one comparison category is assigned. These categories capture logical equivalence, implication relations, contradictions, structural mismatches, and unsupported constructs.

Table 7 summarises the different relations that can be detected. They range from full semantic equivalence to cases where one assertion is stronger or weaker than the other (*Semantic Relation* 5.3.1), as well as situations where assertions are missing, unexpected, incomparable, or cannot be translated within the supported fragment (*Structural Cases* 5.3.2).

Tabelle 7: Logical relations (if applicable) between exemplary expected (φ_E) and generated (φ_G) assertions. *Unsupported* refers to limits of the translation pipeline described in Section 5.1.

Relation	**Logical Condition**	**Expected**	**Generated**
Identical	$\varphi_E = \varphi_G$	`x > 0`	`x > 0`
Empty	–	None	None
Equivalent	$\models \varphi_E \leftrightarrow \varphi_G$	`x > 0`	`0 < x`
Stronger	$\models \varphi_G \rightarrow \varphi_E \ \wedge \ \not\models \varphi_E \rightarrow \varphi_G$	`x >= 0`	`x > 0`
Weaker	$\models \varphi_E \rightarrow \varphi_G \ \wedge \ \not\models \varphi_G \rightarrow \varphi_E$	`x > 0`	`x >= 0`
Dual	$\models \varphi_G \leftrightarrow \neg\varphi_E$	`x > 0`	`x <= 0`
Missing	–	`x > 0`	None
Unexpected	–	None	`x > 0`
Incomparable[6]	$\not\models \varphi_E \rightarrow \varphi_G \ \wedge \not\models \varphi_G \rightarrow \varphi_E$	`x > 0`	`y > 0`
Unsupported	–	`x > 0`	`x.toString() > 0`

Example. Circling back to the example from the beginning of this section, the subsequent explanations become clearer. Let φ_1 and φ_2 denote the respective Java assertions translated into FOL.

A_1 :
```
assert (args[1] == null) == false &&
args[1].contains(null) || args[1] == null;
```
A_2 :
```
assert args[1].contains(null) || args[1] == null;
```

Their logical encodings (after step 1 to 6) are

$$\varphi_1 = (\neg(x = null) \wedge contains(x, null)) \vee (x = null)$$

$$\varphi_2 = contains(x, null) \vee (x = null).$$

[6]The *Incomparable* category is evaluated after checking the *Identical, Equivalent, Stronger, Weaker and Dual* relation.

As shown in the beginning of Section 5, φ_1 and φ_2 are logically equivalent. Hence,

$$\models \varphi_1 \leftrightarrow \varphi_2,$$

and the two assertions are semantically equivalent.

5.3.1 Semantic Relations

The categories *Identical*, *Equivalent*, *Dual*, *Stronger*, *Weaker* and *Incomparable* apply when both the expected assertion φ_E and the generated assertion φ_G are successfully translated into FOL formulas and are not empty. In this case, their relationship is determined purely by logical implication under standard first-order logic semantics.

Identical. Two assertions are classified as *Identical* if their logical representations are syntactically equal:

$$\varphi_E = \varphi_G.$$

This implies immediate semantic equivalence.

Equivalent. Two assertions are *Equivalent* if

$$\models \varphi_E \leftrightarrow \varphi_G.$$

Although syntactically different, they impose the same constraints in all models.

The distinction between *Identical* and *Equivalent* is particularly relevant when comparing automated assertion generation approaches. While both classifications indicate a semantically correct generated assertion, the *Identical* category suggests that the tools not only capture the same semantics but also produce the same logical structure.

Dual. Two assertions are *Dual* if

$$\models \varphi_G \leftrightarrow \neg\varphi_E.$$

They describe complementary conditions. In this case, the generated assertion represents the logical negation of the expected one. When comparing automated assertion generation tools, this classification is particularly informative, as it may indicate that the tools interpret the intended program behavior differently. For instance, one tool may encode the condition under which an assertion should hold, whereas another may encode the complementary condition, such as the circumstances under which an exception should be thrown. Thus, the *Dual* category highlights systematic semantic divergence rather than mere syntactic variation.

Stronger. The generated assertion is classified as *Stronger* if

$$\models \varphi_G \rightarrow \varphi_E \quad \text{and} \quad \not\models \varphi_E \rightarrow \varphi_G.$$

In this case, every model satisfying φ_G also satisfies φ_E, but not vice versa.

Example (Stronger). Let

$$\varphi_E := A \qquad \varphi_G := A \wedge B.$$

We analyze all possible truth assignments:

A	B	$\varphi_E = A$	$\varphi_G = A \wedge B$	$\varphi_E \to \varphi_G$	$\varphi_G \to \varphi_E$
T	T	T	T	T	T
T	F	T	F	F	T
F	T	F	F	T	T
F	F	F	F	T	T

From the table we observe:
- Whenever φ_G is true, φ_E is also true. - However, there exists a valuation (e.g., $A = \mathrm{T}, B = \mathrm{F}$) where φ_E is true but φ_G is false.

Therefore,

$$\models \varphi_G \to \varphi_E \quad \text{and} \quad \not\models \varphi_E \to \varphi_G.$$

Hence, φ_G is strictly stronger than φ_E.

Weaker. The generated assertion is classified as *Weaker* if

$$\models \varphi_E \to \varphi_G \quad \text{and} \quad \not\models \varphi_G \to \varphi_E.$$

In this case, every model satisfying φ_E also satisfies φ_G, but not vice versa.

This classification is defined purely with respect to the logical model induced by the FOL translation and does not necessarily imply that the expected Java assertion is semantically stronger or weaker than the generated one with respect to the original program semantics; this distinction is discussed further in Section 8.1.

Incomparable. Two assertions are classified as *Incomparable* if neither equivalence nor implication holds between them, that is,

$$\not\models \varphi_E \to \varphi_G \quad \text{and} \quad \not\models \varphi_G \to \varphi_E.$$

In this case, the assertions constrain different aspects of the state space and no ordering relation can be established under logical entailment. The Incomparable category is evaluated only after excluding Identical, Equivalent, Stronger, Weaker, and Dual cases.

Example (Incomparable). Let the expected assertion be

A_E : `assert isNull(obj);`

and the generated assertion be

A_G : `assert obj == null;`

Assume that method calls are represented as uninterpreted function symbols in the FOL translation. Then their logical encodings are

$$\varphi_E = isNull(obj) \qquad \varphi_G = (obj = null).$$

Since no axioms connect the uninterpreted predicate $isNull(\cdot)$ with the equality $(obj = null)$, neither implication holds:

$$\not\models \varphi_E \rightarrow \varphi_G \quad \text{and} \quad \not\models \varphi_G \rightarrow \varphi_E.$$

Hence, the assertions are classified as *Incomparable* within the logical model. However, the actual Java implementation [19] reveals that their intended semantics coincide:

```
/* Returns true if the provided reference is null otherwise returns false. */
public static boolean isNull(Object obj) {
    return obj == null;
}
```

Such cases could be addressed by extending the translation pipeline with additional canonicalization rules. For instance, a syntactic normalization rule of the form

- **(SN9) Null Check Canonicalization**

  `isNull(e)` $\Rightarrow$ (`e == null`)

would eliminate this discrepancy and result in a classification as *Equivalent*.

5.3.2 Structural Cases

The categories *Empty*, *Missing*, *Unexpected*, and *Unsupported* apply when at least one of the assertions φ_E or φ_G cannot be meaningfully compared at the semantic level. This occurs either because no assertion is present (e.g., one side is empty) or because the assertion contains language constructs that are not supported by the translation pipeline and therefore cannot be represented as an FOL formula.

Empty. Two assertions are classified as *Empty* if neither the expected nor the generated assertion is present. Although no semantic relation is established, this category may indicate that both approaches agree on the absence of a constraint, or that neither approach produced an assertion for the given context.

Missing. A pair of assertions is classified as *Missing* if an expected assertion is present, but no corresponding generated assertion exists.

This category may indicate that the generation approach does not capture a constraint present in the reference specification, or that it intentionally omits such assertions according to its design.

Unexpected. Analogous to *Missing*, a pair of assertions is classified as *Unexpected* if a generated assertion is present while no expected assertion exists.

This case may indicate that the generation approach introduces an additional constraint not present in the reference specification, or that it applies a broader interpretation of the intended behavior.

Unsupported. A pair of assertions is classified as *Unsupported* if at least one assertion cannot be translated into an FOL formula due to limitations or failures of the translation pipeline.

This situation may arise when the assertion contains language constructs that are outside the supported Java fragment or require semantic information not captured by the current logical model. It may also result from technical issues during translation, such as unresolved type inconsistencies or other internal processing errors.

In such cases, no meaningful logical comparison can be performed. The *Unsupported* category therefore reflects a limitation of the formalisation or its implementation rather than a semantic property of the compared assertions.

Taken together, the above categories form a structured comparison framework for analyzing relationships between expected and generated assertions. By separating semantic relations from structural cases, the evaluation distinguishes logical entailment within the formal model from artefacts introduced by abstraction, modelling decisions, or implementation limitations. This systematic classification enables a fine-grained analysis of assertion generation approaches and provides the foundation for the empirical evaluation presented in the following chapter.

5.4 SMT Solver

To operationalise the formalisation introduced in Sections 5.1 and 5.3, we employ the SMT solver *Z3* [9] as the automated reasoning backend.

After translating Java assertions into FOL and applying logical normalisation, the resulting formulas are encoded into the input language supported by Z3. The solver is then used to decide the satisfiability of counterexample formulas arising from the semantic classification.

Semantic Classification via Counterexample Checks Let A denote the reference assertion and B the generated assertion. Each semantic relation is checked by reducing it to the unsatisfiability of a suitable counterexample formula.

(1) Equivalence

Two formulas are equivalent if there is no model in which they differ. A difference between A and B is captured by the exclusive-or:

$$A \oplus B.$$

Since

$$A \leftrightarrow B \;\equiv\; \neg(A \oplus B),$$

the formulas are equivalent exactly when $A \oplus B$ is false in all models.

Thus, we check

$$\text{unsat}(A \oplus B).$$

If this formula is unsatisfiable, then no model exists where exactly one of A or B holds. Hence, A and B are true in exactly the same models, and therefore equivalent. Syntactic equality (Identical) is checked prior to invoking the SMT solver, as it can be determined directly on the canonical FOL representation.

(2) Dual

Two formulas are dual if one holds exactly when the other does not. That is, they are logical complements:

$$A \leftrightarrow \neg B.$$

A difference from this relation is captured by the exclusive-or:

$$A \oplus \neg B.$$

Since

$$A \leftrightarrow \neg B \;\equiv\; \neg(A \oplus \neg B),$$

the formulas are dual exactly when $A \oplus \neg B$ is false in all models.

Thus, we check

$$\text{unsat}(A \oplus \neg B).$$

(3) Stronger

The generated assertion B is stronger than the reference assertion A if whenever B holds, A must also hold. Equivalence is ruled out beforehand to ensure strictness of the relation. Formally, we check

$$B \rightarrow A.$$

A counterexample to this implication is a model in which B holds but A does not:

$$B \wedge \neg A.$$

Since

$$B \rightarrow A \;\equiv\; \neg(B \wedge \neg A),$$

the implication holds exactly when $B \wedge \neg A$ is unsatisfiable.

Thus, we check

$$\text{unsat}(B \wedge \neg A).$$

(4) Weaker

The generated assertion B is weaker than the reference assertion A if whenever A holds, B must also hold. Formally, we check

$$A \rightarrow B.$$

A counterexample to this implication is a model in which A holds but B does not:

$$A \wedge \neg B.$$

Since

$$A \rightarrow B \;\equiv\; \neg(A \wedge \neg B),$$

the implication holds exactly when $A \wedge \neg B$ is unsatisfiable.

Thus, we check

$$\text{unsat}(A \wedge \neg B).$$

(5) Incomparable

The assertions are classified as incomparable if none of the previous conditions (1)–(4) holds. Concretely, this means that the following formulas are all satisfiable:

$$A \oplus B, \qquad A \oplus \neg B, \qquad B \wedge \neg A, \qquad A \wedge \neg B.$$

Thus,

- A and B are not equivalent,
- they are not logical complements,
- $B \Rightarrow A$ does not hold, and
- $A \Rightarrow B$ does not hold.

Consequently, there exists a model in which A holds but B does not, and another model in which B holds but A does not. The assertions therefore cannot be ordered by logical strength and are classified as logically incomparable.

Z3 is well suited for this task because it provides efficient decision procedures for the theories required by our encoding, in particular linear integer arithmetic, uninterpreted functions, and Boolean logic. Moreover, its incremental solving capabilities allow multiple related queries to be checked efficiently within a single solver context.

Thus, Z3 constitutes the execution layer of our evaluation approach: while the preceding pipeline stages define the formal transformation and comparison semantics, the SMT solver performs the actual logical reasoning required to classify assertion relationships.

However, the use of Z3 does not guarantee a definite result in all cases. The theoretical implications of this limitation are discussed in the following section.

5.5 Decidability and Solver Completeness

The translation pipeline introduced in Section 5.1 maps raw Java assertions to logical formulas in FOL, which capture their semantic meaning. This formalisation enables the comparison of assertions by reducing semantic relations to logical decision problems that are solved using an SMT solver.

However, the underlying decision problem is not decidable in general. Full first-order logic is undecidable (Church's Theorem) [22]. Since our encoding is expressed in first-order logic and includes non-linear arithmetic operators, the overall satisfiability problem is, in theory, undecidable.

As a consequence, the SMT solver is not guaranteed to return a definite answer (`sat` or `unsat`) for every input formula. In particular, it may return `unknown` if it cannot determine satisfiability within its decision procedures.

If this occurs during assertion comparison, none of the semantic relations defined in Section 5.3 can be conclusively established. In this case, the classification result is considered *undetermined*, reflecting solver incompleteness rather than a semantic relation between the assertions.

In practice, modern SMT solvers such as Z3 employ highly optimised decision procedures and heuristics, and often succeed even on theoretically undecidable fragments. The behavior of the solver in our evaluation is discussed in Section 6.5.